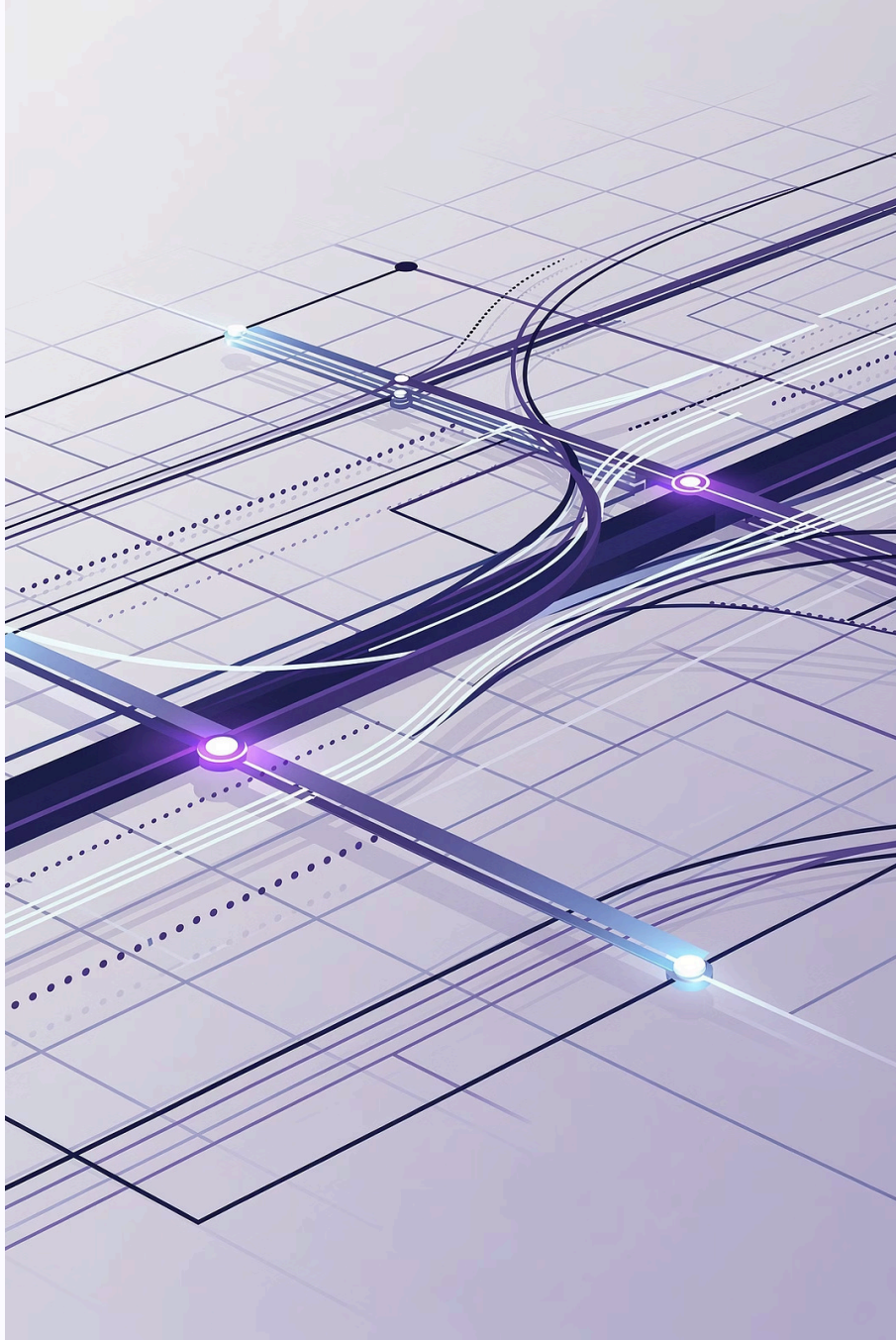
Data Types & Schema in Databricks

A practical guide to understanding, defining, and enforcing schema in Apache Spark and Databricks — the foundation of reliable, high-performance data pipelines.

DATA ENGINEERING | DATABRICKS | APACHE SPARK



Why Schema Matters

Schema is not just metadata — it is a contract between your data and your pipeline. Getting it right from the start prevents costly errors downstream and ensures your data can be trusted at scale.

⚡ Performance

Spark uses schema information to optimise query execution plans, prune columns early, and avoid full scans. Explicit schemas eliminate the overhead of inference jobs at runtime.

✅ Data Quality

Type constraints and nullability rules act as guardrails. Mismatched types surface immediately at ingestion rather than silently corrupting aggregations or ML features.

🔗 Compatibility

Consistent schema across environments ensures that staging, production, and downstream consumers — dashboards, APIs, ML models — all speak the same data language.

Schema Inference vs. Explicit Schema

Spark can infer schema automatically, which is convenient for exploration — but in production pipelines, explicit schema definition is almost always the safer and more performant choice.

Approach	How It Works	Key Risk
Schema Inference	Spark samples data at read time to guess types	Slow on large datasets; types can be guessed incorrectly (e.g. integers inferred as strings); schema changes silently break pipelines
Explicit Schema	Developer defines `StructType` / DDL string upfront	Slightly more upfront effort; but guarantees correctness, faster reads, and predictable failures on bad data

Best Practice: Always prefer explicit schema in production ingestion jobs. Use `enforceSchema=True` or pass a `StructType` directly to `spark.read`. Reserve inference for interactive exploration in notebooks only.

Primitive Data Types

Primitive types are the atomic building blocks of every Spark schema. Choosing the right type is critical — not just for correctness, but for memory efficiency and safe downstream casting.

1

StringType

Stores any text value. Flexible but memory-heavy. Avoid using as a catch-all for numeric or date fields — always cast to the correct type at ingestion.

2

IntegerType / LongType

Use `IntegerType` for 32-bit integers and `LongType` for 64-bit. Choosing too small a type for large IDs causes overflow errors silently.

3

BooleanType

Stores true/false values. Watch out for source systems that encode booleans as `"Y"/"N"` or `1/0` strings — explicit casting is required.

4

TimestampType

Timezone-aware datetime. Essential for event-time processing. Always normalise to UTC at ingestion to avoid timezone-related inconsistencies across regions.

Key Principle: Prioritise **correctness first, then casting**. Read raw data as strings if unsure, validate, then cast explicitly using `cast()` or `to_timestamp()`.

Complex Data Types

Modern data — especially from APIs, event streams, and document stores — rarely arrives flat. Spark's complex types let you model and process nested structures natively, without flattening everything upfront.

StructType

A named collection of fields — essentially a nested row within a row. Common in JSON payloads where objects are embedded inside objects. Access fields using dot notation.

ArrayType

An ordered list of elements of a single type. Frequently seen in event data (e.g. a list of product IDs in an order). Use `explode()` to flatten arrays into individual rows.

MapType

A collection of key-value pairs. Useful for dynamic attributes or properties where keys are not known ahead of time — common in user metadata and feature stores.

Where you'll see these most: JSON ingestion pipelines and Kafka/Kinesis streaming sources. Mastering complex types is essential for real-world data engineering.

Working with Structs

Structs represent nested objects — a row within a row. They are the most common complex type when ingesting JSON from REST APIs or CDC event streams.

Dot Notation Access

Access nested fields directly using dot notation in DataFrame expressions or SQL:

```
df.select("user.address.city")

-- or in SQL:
SELECT user.address.city
FROM events
```

Defining a Struct Schema

Define nested structure explicitly using `StructType` and `StructField`:

```
from pyspark.sql.types import *

schema = StructType([
  StructField("user", StructType([
    StructField("id", LongType()),
    StructField("address", StructType([
      StructField("city", StringType()),
      StructField("postcode", StringType())
    ]))
  ]))
])
```

Key Tips

- Use `col("user.*")` to expand all nested fields to top-level columns
- Use `from_json()` to parse a JSON string column into a Struct
- Avoid deeply nested structs where possible — flatten for analytical workloads

Working with Arrays

Arrays hold ordered lists of values within a single column. Two functions cover the vast majority of array operations in Spark: `explode()` and `size()`.

`explode()` — Flatten Arrays into Rows

Transforms each element of an array into a separate row. Essential when you need to join or aggregate on array elements individually.

```
from pyspark.sql.functions import explode

df.select(
  "order_id",
  explode("product_ids").alias("product_id")
)
```

⚠️ *Note: rows with empty or NULL arrays are dropped by default. Use* `explode_outer()` *to preserve them.*

`size()` — Count Array Elements

Returns the number of elements in an array column. Useful for filtering, feature engineering, or data quality checks.

```
from pyspark.sql.functions import size

df.filter(size("product_ids") > 0)
  .withColumn(
    "num_products",
    size("product_ids")
  )
```

✅ `size()` *returns -1 for NULL arrays — always handle NULLs before using size in comparisons.*

Handling NULLs

NULLs are inevitable in real-world data. A clear, intentional strategy for handling them — defined at schema time — prevents silent failures, incorrect aggregations, and misleading analytics results.

Drop Rows with NULLs

Use `df.na.drop()` or `dropna(subset=["col1","col2"])` when a record is unusable without the field. Ideal for required keys like user IDs or event timestamps. Be cautious — you may silently lose significant data volume.

Replace with a Sentinel Value

Use `df.na.fill({"col": value})` to substitute NULLs with a domain-specific indicator (e.g. `-1` for unknown integer IDs, `"UNKNOWN"` for category strings). Document sentinel values clearly in your data catalogue.

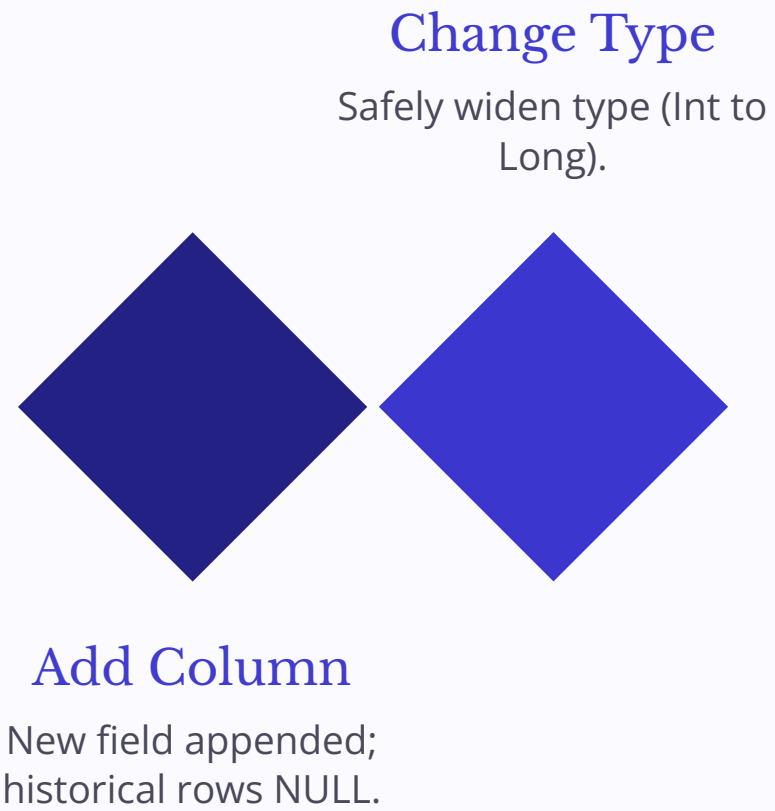
Apply Default Values via Schema

Define `nullable=False` on `StructField` to enforce non-null at read time. Use `coalesce(col, lit(default))` in transformations to fall back to safe defaults without dropping records.

Rule of thumb: Never let NULLs flow silently into aggregations. A `SUM` over a column with unexpected NULLs will undercount. Always audit NULL rates during profiling.

Schema Evolution — A Preview

Real data changes over time. Schema evolution in Delta Lake allows your tables to adapt gracefully — without rewriting the entire dataset or breaking existing consumers. This topic will be covered in depth on Day 4; here is a foundational preview.



Adding a Column

Enable with `spark.databricks.delta.schema.autoMerge.enabled = true` or pass `.option("mergeSchema", "true")` at write time. Historical rows will show `NULL` for the new column.

Changing a Type

Type changes require `overwriteSchema`. Only *widening* casts are safe (e.g. `INT → LONG`). Narrowing casts risk data loss and should be avoided in production without a full migration plan.

Coming up on Day 4: Delta Lake schema enforcement, schema evolution strategies, and time travel for schema rollback.

What's Next: Delta Lake

You now have the foundational tools to define, enforce, and manipulate schema in Databricks with confidence. Correct data types and explicit schema definition are the bedrock of everything that follows.

01

Schema Mastery ✅

Primitive and complex types, NULL handling, schema inference vs. explicit definition — all covered in this session.

02

Reliable Data Management

With schema in place, your pipelines can validate, transform, and store data predictably — the prerequisite for any production-grade workload.

03

Delta Lake — Day 4

Next session: ACID transactions, schema evolution, time travel, and optimised storage with Delta Lake — built directly on the schema foundations from today.

> *"Now that we can define and manage data reliably, we're ready to build on it — welcome to Delta Lake."*